

CS 225 Examination

Data Structures and Algorithms in C++ – Answer Key

1. (A) $O(\log n)$.

In a balanced BST (like AVL or Red-Black tree), the height is $O(\log n)$. Each comparison eliminates roughly half the remaining nodes, leading to logarithmic search time.

2. (A) A shallow copy copies pointer values; a deep copy allocates new memory and copies the data.

Shallow copy duplicates pointers, so both objects point to the same memory. Deep copy creates new memory and copies actual data, ensuring independence. This is critical for classes with dynamically allocated members.

3. (C) Stack.

Function calls follow LIFO (Last In, First Out) semantics: the most recently called function returns first. A stack naturally models this with push for function entry and pop for return.

4. (B) $O(n^2)$.

When the pivot is consistently the smallest or largest element (e.g., already sorted array with first element as pivot), each partition reduces size by only 1, leading to $n + (n - 1) + \dots + 1 = O(n^2)$ comparisons.

5. *Sample answer:*

Amortized analysis: Analyzes the average cost per operation over a sequence of operations, rather than worst-case for a single operation.

Dynamic array example:

- When full, the array doubles in size, requiring copying n elements: $O(n)$ for that insertion
- However, this happens only at sizes 1, 2, 4, 8, 16, ..., i.e., every n insertions
- Total cost for n insertions: $n + (1 + 2 + 4 + \dots + n) < n + 2n = 3n$
- Average (amortized) cost per insertion: $3n/n = O(1)$

The occasional expensive resize is “amortized” over many cheap insertions, yielding constant average time.

6. *Sample answer:*

Min-heap: A binary tree where each parent is smaller than its children. Provides $O(\log n)$ insertion and $O(1)$ access to minimum element, $O(\log n)$ deletion of minimum.

Priority queue: An abstract data type (ADT) that supports insertion and extraction of the highest-priority element. A min-heap is a common implementation.

Dijkstra’s algorithm with priority queue:

- Use min-heap to always extract the vertex with smallest tentative distance
- Insert/update: $O(\log V)$ per operation
- Each vertex inserted/extracted once, each edge causes potential decrease-key
- Time complexity: $O((V + E) \log V)$ with binary heap, or $O(V \log V + E)$ with Fibonacci heap

7. *Sample answer:*

Rule of Three: If a class manages resources (e.g., dynamic memory), define:

- (a) Destructor (to free resources)
- (b) Copy constructor (for deep copying)
- (c) Copy assignment operator (for deep assignment)

This prevents double-deletion and dangling pointers.

Rule of Five (C++11): Also define:

- (a) Move constructor (for efficient transfer of resources)
- (b) Move assignment operator

This enables move semantics for performance optimization.

Rule of Zero: Prefer using RAII classes (`std::unique_ptr`, `std::vector`) that manage resources automatically. If no explicit resource management is needed, don't define any of the special member functions—let the compiler generate them. This is the safest and most maintainable approach.

4. *Sample answer:*

`std::vector`:

- **Advantages:** Contiguous memory (cache-friendly), fast random access $O(1)$, good iteration performance
- **Disadvantages:** Insertion/deletion in middle is $O(n)$, reallocation on growth
- **Use when:** Random access needed, sequential iteration, append-heavy workload

`std::list` (doubly-linked list):

- **Advantages:** $O(1)$ insertion/deletion at any position (if iterator available), no reallocation, stable iterators
- **Disadvantages:** No random access ($O(n)$ to reach element), poor cache locality, higher memory overhead (pointers per node)
- **Use when:** Frequent insertions/deletions in middle, need iterator stability, splicing operations

General guideline: Default to `std::vector` due to cache efficiency. Use `std::list` only when insertion/deletion patterns specifically benefit from $O(1)$ operations at known positions.